

# Testing Black Box e White Box

Trentin Tommaso

December 26, 2025

## Contents

|          |                                                         |          |
|----------|---------------------------------------------------------|----------|
| <b>1</b> | <b>Testing black box</b>                                | <b>2</b> |
| 1.1      | Classi di equivalenza . . . . .                         | 2        |
| 1.1.1    | Suddivisione . . . . .                                  | 2        |
| 1.2      | Boundary values . . . . .                               | 2        |
| 1.2.1    | Problemi . . . . .                                      | 2        |
| <b>2</b> | <b>Testing white box</b>                                | <b>2</b> |
| <b>3</b> | <b>Control Flow Graph (CFG)</b>                         | <b>3</b> |
| 3.1      | Idea . . . . .                                          | 3        |
| 3.1.1    | Nodi predicato . . . . .                                | 3        |
| 3.1.2    | Code to CFG . . . . .                                   | 3        |
| 3.2      | Criteri di copertura . . . . .                          | 3        |
| 3.2.1    | Copertura dei comandi (Statement coverage) . . . . .    | 4        |
| 3.2.2    | Copertura delle decisioni (Decision coverage) . . . . . | 4        |
| 3.2.3    | Decisioni vs condizioni . . . . .                       | 4        |
| 3.2.4    | Condition coverage . . . . .                            | 4        |
| 3.2.5    | Copertura di condizioni E decisioni . . . . .           | 4        |
| 3.3      | Esempio . . . . .                                       | 5        |

# 1 Testing black box

Sistema appare come una "scatola nera" (struttura interna non visibile al tester), approccio particolarmente utile quando non si ha accesso a codice sorgente. Consiste nell'esercitare il sistema immettendo input e osservando valori degli output.

- Sfrutta conoscenza di:
  - Interfaccia del sistema;
  - Documentazione (es. descrizione degli scenari).
- Non tiene conto di:
  - Codice sorgente;
  - Stato interno dell'applicazione.

Gli obiettivi principali del testing black box possono essere la *copertura degli scenari di esecuzione* definiti nei casi d'uso o la *copertura delle funzionalità* definite nella specifica dei requisiti, tuttavia lo spazio dei possibili input è estremamente ampio e ne va quindi scelto un sottoinsieme significativo.

## 1.1 Classi di equivalenza

In base ai requisiti del SW richiesto si suddivide lo spazio degli input in sottoinsiemi detti **classi di equivalenza** i cui elementi (idealmente) si comporteranno allo stesso modo durante l'esecuzione. I casi di test eseguiranno poi *almeno un test per ogni classe di equivalenza*.

### 1.1.1 Suddivisione

Anche spazio dell'output può essere diviso in partizioni di equivalenza in cui il sistema avrà comportamento simile. Si può considerare la caratteristica di **validità** dell'input a esercitare una particolare funzionalità del SW per definire 1+ classi di **input validi** e 1+ di **input non validi**.

## 1.2 Boundary values

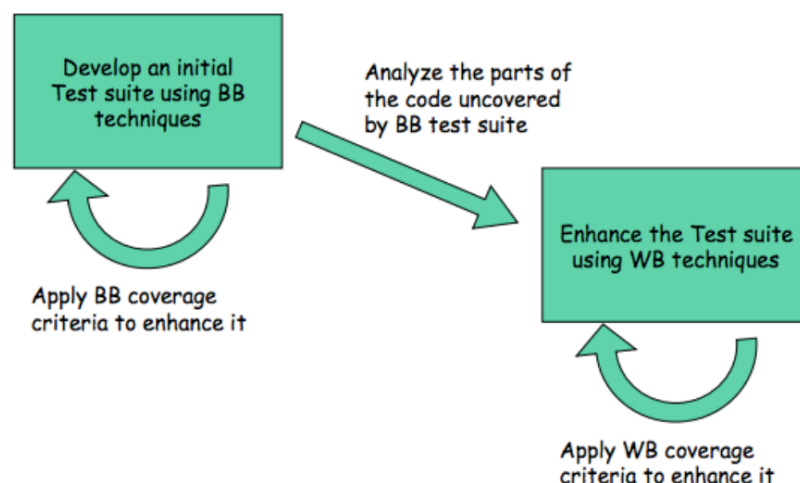
Programmatore fanno spesso errori per valori di input vicini ai confini (boundaries) delle classi di equivalenza. Il SW va quindi testato anche con valori di input che coincidono (o sono vicini) ai valori limite delle classi di equivalenza.

### 1.2.1 Problemi

A volte non è possibile determinare a priori input per coprire le classi di equivalenza ma bisogna comunque conoscere le precondizioni, in tal caso va specificata la **precondizione** in modo da tenere conto di quali valori per le variabili sono accettabili.

# 2 Testing white box

Utilizza struttura interna del SW per ricavare dati di test, può fornire maggiori info al debugger sulla posizione dell'errore e complementa il testing black box in quanto controlla le parti di programma non testate da esso che potrebbero contenere errori.



## 3 Control Flow Graph (CFG)

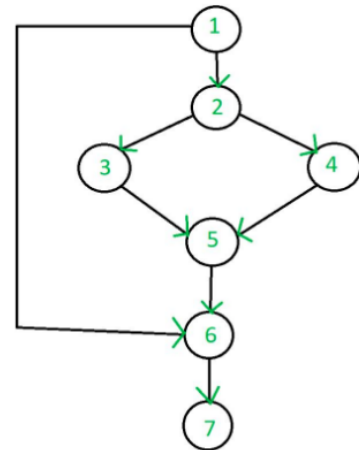
### 3.1 Idea

Grafo orientato che rappresenta il trasferimento del flusso di controllo tra blocchi di istruzioni di un programma P, contiene:

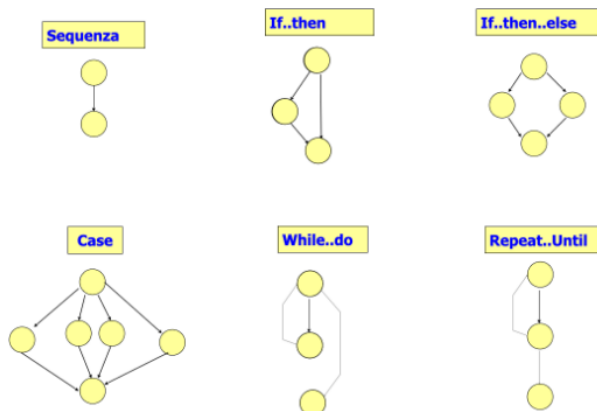
- **nodi** che rappresentano blocchi di 1+ istruzioni, tutte eseguite se il flusso di controllo raggiunge la prima di esse;
- **archi** che rappresentano trasferimento del flusso di controllo tra coppia di nodi che esso collega;
- **nodo di ingresso** e **nodo d'uscita**

I nodi possono essere:

- **istruzione** con 1 solo arco uscente;
- **predicato** che corrispondono a condizioni e hanno 2 archi uscenti corrispondenti a V/F o 3+ archi uscenti in caso di switch-case.



#### 3.1.1 Nodi predicato

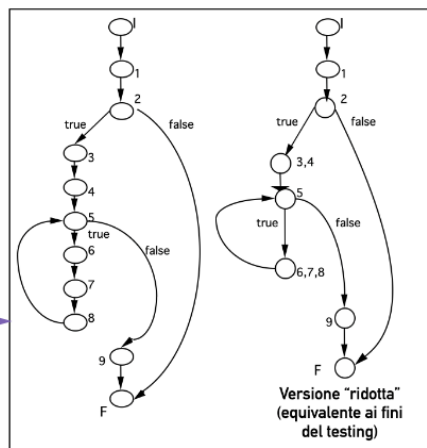


IL CICLO FOR PUÒ ESSERE RAPPRESENTATO COME UN WHILE-DO AGGIUNDO UN NODO PER L'INIZIALIZZAZIONE DELLA VARIABILE E UN ALTRO PER L'INCREMENTO

#### 3.1.2 Code to CFG

```
int Quadrato (int x){
1.  read(x);
2.  if (x > 0){
3.      n = 1;
4.      y = 1;
5.      while (x > 1) {
6.          n = n + 2;
7.          y = y + n;
8.          x = x - 1;
9.      }
10. write(y);
11. }
```

LE ISTRUZIONI ESEGUITE SEMPRE IN SEQUENZA POSSONO ESSERE RAGGRUPPATE IN UN UNICO BLOCCO  
ATTENZIONE AI RAGGRUPPAMENTI: 5 È UN PREDICATO E NON PUÒ ESSERE RAGGRUPPATO CON LE ISTRUZIONI 3 E 4

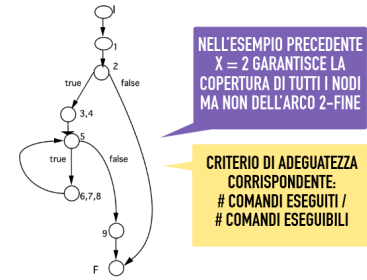


## 3.2 Criteri di copertura

Fondati sull'adozione di metodi di Coperturaa degli oggetti che compongono struttura dei programmi, vanno definiti un insieme di casi di test aventi dati in input in grado di **coprire** tutti i componenti di interesse del SW almeno una volta durante la loro esecuzione.

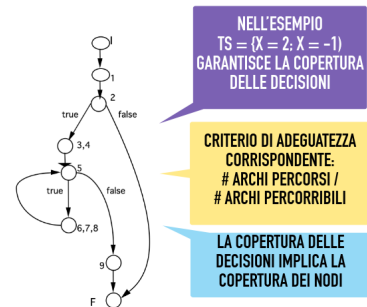
### 3.2.1 Copertura dei comandi (Statement coverage)

Richiede di coprire tutti i nodi del CFG durante il testing ed eseguire tutte le istruzioni, il che dà una debole garanzia di scoprire i difetti essendo abbastanza semplice da soddisfare (non assicura di coprire sia il ramo *true* che il ramo *false* delle decisioni).



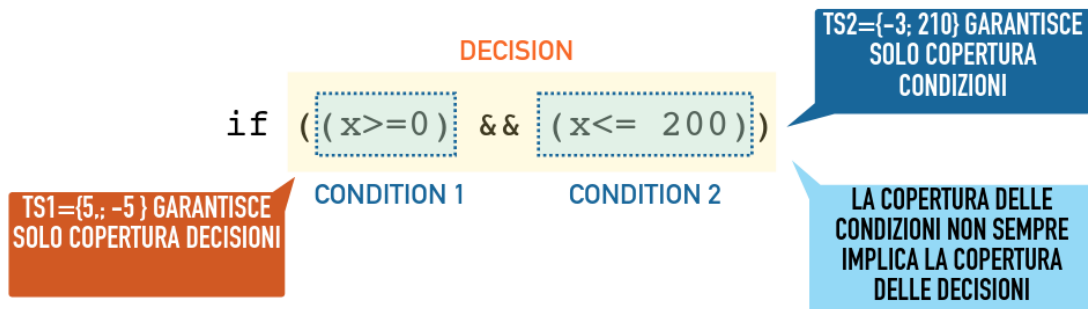
### 3.2.2 Copertura delle decisioni (Decision coverage)

Richiede che ciascun arco del CFG sia attraversato almeno una volta, ogni decisione del programma deve quindi assumere sia valore *true* che valore *false* durante il testing. Unico limite in caso di decisioni in cui vengono valutate più condizioni.



### 3.2.3 Decisioni vs condizioni

- **Decisione:** predicato di una istruzione condizionale o di una iterativa;
- **Condizione:** espressione atomica booleana che appare in una decisione.

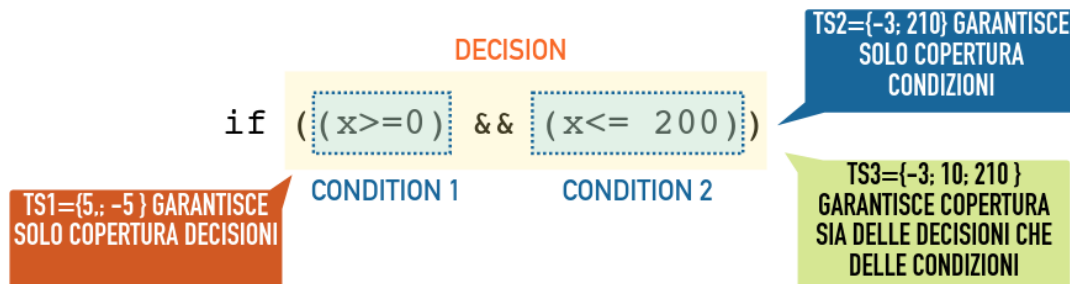


### 3.2.4 Condition coverage

Richiede che ciascuna condizione dei nodi decisione di un CFG debba essere valutata per valori sia *true* che *false*.

### 3.2.5 Copertura di condizioni E decisioni

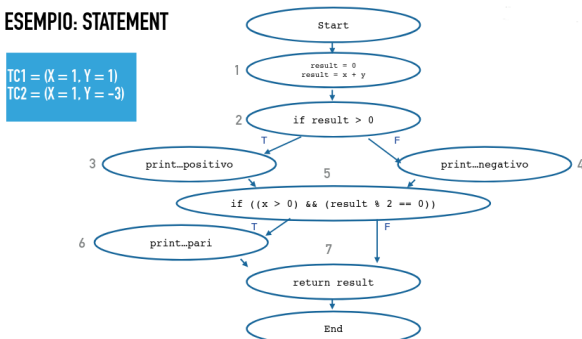
Criterio più forte perché assicura che casi di test coprono tutte le condizioni ma anche tutte le decisioni.



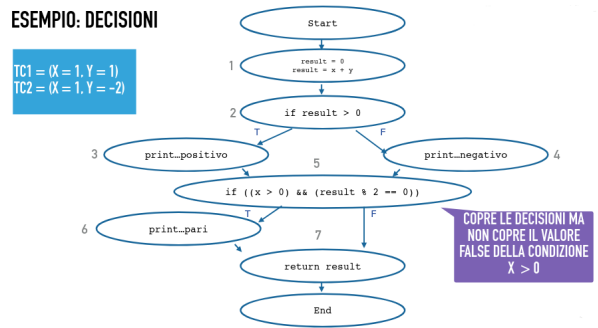
### 3.3 Esempio

```
def esempio_control_flow(x, y):
    # N1 - Blocco di comandi 1: Assegnamento
    result = 0
    result = x + y
    # N2 - Decisione 1: Verifica se il risultato è positivo o negativo
    if result > 0:
        # N3 - Comando 2: Stampa il risultato se positivo
        print("Il risultato è positivo:", result)
    else:
        # N4 - Comando 3: Stampa un messaggio se il risultato è non positivo
        print("Il risultato non è positivo:", result)
    # N5 - Decisione 2: Verifica se il risultato è pari o dispari
    if ((x > 0) && (result % 2 == 0)):
        # N6 - Comando 4: Stampa un messaggio se il risultato è pari
        print("Il risultato è un numero pari e il primo addendo positivo.")
    # N7 - Comando 6: Restituisce il risultato
    return result
```

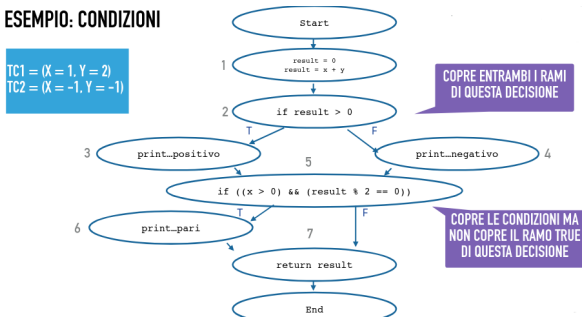
ESEMPIO: STATEMENT



ESEMPIO: DECISIONI



ESEMPIO: CONDIZIONI



ESEMPIO: CONDIZIONI E DECISIONI

